

Administration des Systèmes

MODULE 6 : GESTION DES PAQUETS

HÉDI NSIBI – FACULTÉ DES SCIENCES DE MONASTIR 2025 / 2026

Définition d'un paquet logiciel

- Un **paquet logiciel** est une archive contenant tous les éléments nécessaires à l'installation d'un programme.

Il contient :

- **Les fichiers binaires exécutables** (le programme compilé)
- **Les bibliothèques** nécessaires à son fonctionnement
- **Les fichiers de configuration**
- **Les scripts de post-installation / suppression** (pour automatiser certaines tâches)
- **Les métadonnées** : nom, version, dépendances, architecture, mainteneur, etc.

NB: un paquet n'est pas un simple fichier exécutable, mais une structure complète assurant une installation cohérente et traçable sur le système.

Paquet source vs paquet binaire

Type de paquet	Contenu	Utilisation	Exemple
Paquet binaire	Programme déjà compilé	Installation rapide et stable	htop.deb, nginx.rpm
Paquet source	Code source + scripts de compilation	Personnalisation, adaptation, recompilation	nginx-1.24.tar.gz

Paquet binaire

- Un **gestionnaire de paquets** est un outil qui permet d'**installer, mettre à jour et supprimer des logiciels automatiquement**, sans passer par les étapes manuelles de compilation.
- Objectif:

Simplifier la vie de l'administrateur en gérant :

- Les **dépendances** (bibliothèques nécessaires)
- Les **mises à jour**
- Les **signature de sécurité**
- La **désinstallation propre**

Exemples connus :

Distribution	Gestionnaire principal	Commandes typiques	Format du paquet
Debian / Ubuntu	APT	apt install, apt update	,deb
Fedora / CentOS / RHEL	DNF (anciennement YUM)	dnf install, dnf update	.rpm
OpenSUSE	Zypper	zypper install	.rpm
Arch Linux	Pacman	pacman -S	.rpm

Avantages des gestionnaires

- Vérification automatique des signatures (sécurité)
- Gestion simplifiée des dépendances
- Installation rapide depuis des dépôts officiels
- Mise à jour centralisées du systèmes

- **Exemple** :

Sudo apt install htop

➞ Une seule commande : le système télécharge, compile (si besoin), installe, et configure le logiciel automatiquement.

Formats standards (.deb et .rpm)

- Formats les plus répandus :
- .deb → Debian, Ubuntu, Linux Mint, etc.
- .rpm → Red Hat, CentOS, Fedora, openSUSE
- Ces formats ne sont **pas interchangeables**
- Chaque format dispose de ses outils dédiés:
- .deb → dpkg, apt
- .rpm → rpm, dnf, yum

Cycle de vie d'un paquet

- **Téléchargement**

→ Récupération du paquet depuis un dépôt ou une source externe(apt download, wget, etc)

- **Installation**

→ Décompression et copie dans des fichiers dans le système (apt install, dnf install, rpm -i)

- **Mise à jour**

→ Remplacement propre d'une version par une autre (apt upgrade, dnf update)

- **Suppression /purge**

→ Désinstallation du paquet et de ses fichiers associés (apt remove, rpm - e)

Objectif : assurer une gestion cohérente, stable et traçable des logiciels installés.

Gestion des dépendances et cohérence du système

- **Dépendance**: Un paquet nécessite d'autres paquets pour fonctionner

→ Exemple: vlc dépend de libavcodec et libqt5gui

- **Gestion automatique:**
  - Téléchargement des dépendances manquantes
  - Vérification de compatibilité des versions
- **Cohérence du système:**
  - Prévention des conflits de version
  - Suppression propre sans casser les autres paquets
- **Outlis clés**
  - apt-get –f install (Debian/Ubuntu)
  - dnf check /yum check (Redhat/Fedora)

Installation par le code source (1/10)

- **Objectif** : Comprendre pourquoi et quand on installe depuis les sources.
- **Points clés :**

- Flexibilité, personnalisation, compatibilité avec le matériel
- Cas d'usage: absence de paquet précompilé, version récentes.
- Inconvénients: plus long, erreurs possibles

Installation par le code source (2/10)

- **Les fichiers essentiels à la compilation**

- **Configure** → détecte l'environnement.
- **Makefile** → définit les règles de compilation.
- **CMakelists.txt** → alternative moderne via CMake.
  - Générer automatiquement un make_file propre
  - Gérer les dépendances
  - Détecter la configuration système
  - Rendre le projet portable(linux, windows, macOs)

Exemples

```
sudo apt update
sudo apt install build-essential libncurses5-dev libncursesw5-dev
```

```
wget https://github.com/htop-dev/htop/archive/refs/tags/3.3.0.tar.gz
```

```
tar -xvzf 3.3.0.tar.gz
cd htop-3.3.0
```

```
./configure
```

```
make
```

```
sudo make install
```

Installation par le code source (3/10)

- **Pré-requis et outils nécessaires**
- **Outils indispensable:**

- gcc, g++, make, autoconf, automake, libtool, build-essential
- Vérifier leurs présence **(gcc –version, make --version)**
- Installer les paquets requis selon la distribution (apt/dnf)

Installation par le code source (4/10) – Etapes classiques

- Télécharger le code source:

```
wget https://exemple.com/logiciel.tar.gz
```

- Extraire les fichiers

```
tar -xvzf logiciel.tar.gz
cd logiciel/
```

Installation par le code source (5/10) – Configuration du projet

- Commande :

```
./configure
```

- Paramètres courants:

- **--prefix=/usr/local**: répertoire d'installation
- **--enable-feature** / **--disable-feature**

Vérification des dépendances et génération du **Makefile**

Installation par le code source (6/10) – Compilation

- **Lancer la compilation**
- make
- **Explication du processus:**
- Traduction du code source en binaire.
- Génération d'exécutables ou de librairies
- Gestion des erreurs et options (make –j4 pour le multithreading)

Installation par le code source (7/10) – Installation

- Installation avec privilèges administrateur :

```
sudo make install
```

- Vérification

```
which logiciel
logiciel --version
```

- Fichiers installés dans /usr/local/bin, /usr/local/lib

Installation par le code source (8/10) – Désinstallation

- Si supporté

```
sudo make uninstall
```

- Sinon suppression manuelle
- Utiliser find ou whereis pour localiser les fichiers installés

Comparaison : code source vs gestionnaire de paquets (9/10)

Critère	**Installation par source**	**Gestionnaire de paquets**
Flexibilité	Très haute	Moyenne
Sécurité	Moins contrôlée	Signatures et vérifications
Facilité de mise à jour	Manuelle	Automatique
Temps d'installation	Long	Rapide

Bonnes pratiques & maintenance (10/10)

- Documenter chaque installation (journal d'admin)
- Sauvegarder les commandes et version utilisées
- Tester avant déploiement en production
- Utiliser /usr/local ou /opt pour isoler les installation
- Nettoyer régulièrement (make clean)

Sécurité & Intégrité : Vérification d'intégrité du code source

- **Objectif**: S'assurer que le fichier télécharger n'a pas été altéré ou corrompu
- **Méthodes:**

- Chaque projet open source publie une **empreinte(Hash)**: SHA256 ou MD5
- On compare la somme du fichier télécharger avec celle fournie sur le site officiel

```
sha256sum logiciel.tar.gz
md5sum logiciel.tar.gz
```

----------**Code source** : utilisé avant la compilation
---------- **Paquet binaire** elle est vérifie automatiquement par le gestionnaire

- Si les valeurs correspondent, le fichier est **authentique et intact**. Sinon, **ne jamais l'exécuter ni le compiler**.

Sécurité & Intégrité : Vérification de la signature GPG

- **Objectif:**

S'assurer que le paquet provient bien du **fournisseur légitime** (authenticité).

- **Principe:**

Le développeur signe le paquet avec sa **clé privée GPG**,
et l'utilisateur vérifie la signature avec la **clé publique** du projet.

```
gpg --verify logiciel.tar.gz.sig logiciel.tar.gz
```

Cas d'usage:

- Il (l'utilisateur)doit importer la clé public du fournisseur(développeur) afin de vérifier la signature du paquet
- Pour un **paquet source**: souvent .asc ou sig accompagant le code
- Pour un **paquet binaire**: apt, dnf et zypper vérifier autmatiquement les clés GPG

Sécurité & Intégrité : Importance en environnement professionnel

- Pourquoi c'est crucial:

Protection contre les paquets compromis(supply chain attrack)

1. Conformité aux politiques de sécurité IT
2. Traçabilité et audibilité des installations
3. Assurance qualité lors de déploiements automatisés

- **Bonne Pratique:**

- Toujours télécharger des sources officielles
- Importer les clés GPG de manière sécurisée
- Ne jamais désactiver la vérification (--nogpgcheck ou –allow-unauthentiificated)

Gestion de base-niveau

- **Gestion bas-niveau (low-level package management)**

La gestion de bas niveau permet d'installer, supprimer ou interroger directement des paquets individuels

Ne gére pas automatiquement les dépendances

- **RPM (Red Hat Package Manager) :** utilisé sur Red Hat, CentOS, Fedora.
- **DPKG (Debian Package) :** utilisé sur Debian, Ubuntu.

Exemple :

- rpm -i package.rpm
- dpkg -i package.deb

Gestion de haut-niveau

- **Gestion haut-niveau (High-Level Package Management)**

La gestion de haut niveau automatise l'installation, la mise à jour et la suppression des paquet avec toutes leurs dépendances

Elle utilise les dépôts officiels pour récupérer les paquets

Exemples de gestionnaires

APT(Debian, Ubuntu)

apt update # mettre à jour la liste des paquets

apt install nginx # installer un paquet avec dépendances

apt upgrade # mettre à jour tous les paquets

YUM/ DNF (Red Hat, CentOS, Fedora

yum install httpd

dnf update

yum remove package

Gestion des paquets avec DNF / YUM

- **Commandes principales :**
- **Installer** : sudo dnf install paquet
- **Supprimer** : sudo dnf remove paquet
- **Mettre à jour** : sudo dnf update
- **Rechercher** : sudo dnf search mot_clef
- **Informations** : sudo dnf info paquet
- **Historique** : sudo dnf history

- **Maintenance** :
- **Nettoyage du cache** : sudo dnf clean all
- **Gestion des dépôts** : fichiers .repo dans /etc/yum.repos.d/

Atelier pratique : DNF / YUM

- **Objectif** : Gérer un dépôt et installer un paquet
- **Tâches** :
- Ajouter le dépôt EPEL(Extra Packages for Entreprise Linux)

```
sudo dnf install epel-release      # pour CentOS/RHEL 8+# ou
sudo yum install epel-release      # pour CentOS/RHEL 7
```

- Installer un paquet depuis ce dépôt (ex: htop)

```
sudo dnf install htop
```

- Vérifier la provenance du paquet

```
dnf info htop       # ou
yum info htop
```

- Supprimer et nettoyer les paquets

```
sudo dnf remove htop
```

Gestion des paquets avec APT

- **Commandes principales :**
- **Mise à jour liste** : sudo apt update
- **Installer** : sudo apt install paquet
- **Supprimer** : sudo apt remove paquet
- **Mise à niveau** : sudo apt upgrade
- **Rechercher** : sudo apt search mot_clef
- **Détails** : sudo apt show paquet
- **Nettoyage** : sudo apt autoremove

Configuration des dépôts APT

- **Fichier principal : /etc/apt/sources.list**
- **Exemple d'entrée :**

deb http://archive.ubuntu.com/ubuntu jammy main restricted universe

- **Ajouter un PPA :**

sudo add-apt-repository ppa:graphics-drivers/ppa

sudo apt update

- **Supprimer un dépôt** : supprimer le fichier du dossier /etc/apt/sources.list.d/

Automatisation et scripts

- **Script Bash d'installation automatique :**

#!/bin/bash

sudo apt update -y

sudo apt install -y curl vim

sudo apt remove -y nano

apt list --installed > liste_paquets.txt

Utile pour **audit, documentation ou sauvegarde de l'état des paquets**.

- **Sauvegarde de la liste des paquets :**

dpkg --get-selections > sauvegarde.txt

Permet de connaître l'état exact des paquets gérés par **dpkg**

- **Restauration :**

dpkg --set-selections < sauvegarde.txt # restaurer la liste des paquets depuis la sauvegarde

sudo apt-get dselect-upgrade #appliquer les changement avec APT, synchroniser le système

Automatiser les mises à jour avec Cron

- **Automatiser les mises à jour hebdomadaires :**
- **Exemple de tâche cron :**

sudo crontab –e

```
# Ajoutez la ligne suivante :
0 2 * * * /usr/bin/apt update && /usr/bin/apt upgrade –y
```

→ **Exécute les mises à jour tous les jours à 02h00.**

- **Avantage** :

Maintenance automatisée du système

Audit, logs et dépannage

- **Fichiers de logs :**
  - /var/log/dpkg.log
  - /var/log/yum.log
  - /var/log/dnf.log
  - **/var/log/apt/history.log** → Historique des actions APT
  - **/var/log/apt/term.log** → Détail des sorties (messages) des commandes APT.

- **Outils de vérification :**
  - rpm -Va : vérifie l'intégrité des paquets RPM
  - dpkg --audit : vérifie les paquets défectueux

- **Problèmes courants :**
  - Cache corrompu → sudo apt clean
  - Fichier verrouillé → sudo fuser -v /var/lib/dpkg/lock

Évaluation finale

- QCM de 10 questions :
- 1. Différence entre APT et DNF
- 2. Fichier de configuration des dépôts APT
- 3. Commande de nettoyage sous DNF
- 4. Rôle de la clé GPG
- 5. Étapes pour ajouter un dépôt PPA
- 6. Commande pour vérifier l'intégrité RPM
- 7. Où se trouvent les logs d'APT ?
- 8. Automatisation des mises à jour : quelle commande ?
- 9. Comment sauvegarder la liste des paquets ?
- 10. Rédigez un script d'installation automatique simple.